

تقرير المرحلة الثالثة – المباني والشقق والأسرّة + موديول التدقيق الحقيقي (Phase 3 – Buildings, Apartments) (Beds & Real Audit)

اسم المرحلة ورقمها

المرحلة الثالثة: المباني والشقق والأسرّة (Buildings, Apartments),
Phase 3 – Beds، وتتضمن أيضًا بندًا أُضيف بعد مراجعة المرحلة الثانية: بناء
موديول audit الحقيقي مبكرًا، وربط endpoint تغيير حالة الـ KYC به بأثر رجعي.

الهدف من المرحلة

بناء الهرم الأساسي لبيانات العقار (Building → Apartment → Bed) التي تعتمد
عليه محركات الحجز (Booking Engine) في المرحلة الرابعة. كل سرير لازم يرجع
لشقة واحدة، والشقة ترجع لمبنى واحد، والمبنى مملوك لصاحب واحد (Owner) —
مع بناء آلية تسجيل صحيحة لكل تغيير في حالة السرير من أول المرحلة.

بالإضافة لذلك، وبناءً على توصية ظهرت في تقرير المرحلة الثانية (أن موديول audit
المركزي كان لسه placeholder فاضي)، تقرر في هذه المرحلة بناء الموديول
الحقيقي بدلًا من الانتظار للمرحلة الخامسة، وربط كل من: (أ) تغييرات حالة السرير،
و(ب) قرارات مراجعة الـ KYC من المرحلة الثانية — بنفس آلية التدقيق الحقيقية.

الملفات والمجلدات التي اتعملت فعليًا

1. موديول المباني (Buildings Module)

src/modules/buildings/

- └─ building.model.js → النموذج: owner_id, name, area, address
- └─ building.repository.js → طبقة الوصول لقاعدة البيانات
- └─ building.service.js → ل/الحذف/القراءة المتداخلة/حساب الإشغال
- └─ building.controller.js → معالجات الطلبات + التحقق من الملكية
- └─ building.routes.js → تعريف المسارات (owner-only)

2. موديول الشقق (Apartments Module)

src/modules/apartments/

- └─ apartment.model.js → النموذج: building (ref), owner_id (de
- └─ apartment.repository.js
- └─ apartment.service.js
- └─ apartment.controller.js
- └─ apartment.routes.js

3. موديول الأسرّة (Beds Module)

src/modules/beds/

- └─ bed.model.js → النموذج: apartment (ref), building (
- └─ bed.repository.js
- └─ bed.service.js → حساب الإشغال + CRUD منطق الـ
- └─ bed.controller.js
- └─ bed.routes.js
- └─ bed-history.service.js → audit.se حول (thin wrapper) غلاف رفيع

4. موديول التدقيق الحقيقي (Audit Module) – مبني بالكامل هذه المرحلة

```
src/modules/audit/  
├─ audit.model.js      → actor, action, entity_type, entity_id, b  
├─ audit.repository.js → create + حذف على الإطلاق  
├─ audit.service.js    → writeAuditLog(), getEntityHistory(), lis  
├─ audit.controller.js → (super-admin) للقراءة فقط  
└─ audit.routes.js
```

5. بنية تحتية مشتركة (جديدة هذه المرحلة)

```
src/shared/utils/  
└─ pagination.util.js → parsePagination() و buildMeta() – كل من
```

6. ملفات معدلة

```
src/app.entry.js      → ربط /api/buildings, /api/apart  
src/modules/kyc/kyc.service.js → updateVerificationStatus() في
```

7. ملفات الاختبار

```
tests/integration/buildings-apartments-beds.test.js → (سم الاختبارات)
```

شرح تفصيلي لكل خطوة اتفدت

الخطوة 1: نموذج المبنى (building.model.js)

owner_id مُعرّف ك String مش ObjectId عمداً، عشان يطابق نوع User.owner_id في auth.model.js (من المرحلة الأولى) — ده بيخلي أي

مقارنة ملكية (req.user.ownerId === building.owner_id) تتم مباشرة من غير أي populate() أو تحويل نوع بيانات. حقل area (المنطقة/الحي) انحط كنص حر مش مبني على حساب مسافة، مطابقةً لنص المرحلة. الفهرسة (Indexes) اتعرّفت مع النموذج مباشرة: owner_id و area، حسب البند 4.1 في CLAUDE.md.

الخطوة 2: نموذج الشقة (apartment.model.js)

كل شقة مرتبطة بمبنى واحد عبر building (ObjectId reference). تم أخذ قرار تقني بنسخ (denormalize) قيمة owner_id من المبنى الأب على مستند الشقة وقت الإنشاء، بدل الاعتماد على populate('building') في كل عملية تحقق من الملكية أو استعلام قوائم — التفاصيل والسبب في قسم "القرارات التقنية" أدناه.

الخطوة 3: نموذج السرير (bed.model.js)

نفس منطق الـ denormalization تطبق هنا كمان: building و owner_id منسوخين من الشقة الأب وقت الإنشاء. حقل status يستخدم enum BED_STATUS الموجود بالفعل في src/config/constants.config.js (من مرحلة سابقة) — available/pending/occupied/maintenance — وليس القيم المذكورة حرفيًا في نص المرحلة (available/requested/confirmed/vacating). هذا انحراف متعمد عن نص الخطوة الأصلية، موضح بالتفصيل في قسم "أي انحراف عن الخطوة الأصلية" أدناه.

الخطوة 4: بناء موديل التدقيق الحقيقي (audit.model.js) +

bed-history.service.js و (audit.service.js)

audit.model.js بُني كمجموعة (collection) عامة وقابلة لإعادة الاستخدام من أي موديل مستقبلي: actor (مين نَقِّذ الفعل)، action (اسم الفعل، نص حر مش enum عشان الموديلات المستقبلية تقدر تضيف أفعال جديدة من غير تعديل النموذج المشترك)، entity_type و entity_id (نوع ومعرّف السجل المتأثر)، و after_state/before_state (لقطة قبل/بعد التغيير، من نوع Mixed عشان الشكل يختلف حسب نوع الكيان). المجموعة دي **append-only بالكامل** — audit.repository.js بيعرّض دالة create فقط بالإضافة لدوال قراءة، وما فيهاش أي دالة update أو delete على الإطلاق، فمن المستحيل بنيويًا إن أي كود

في المشروع يعدّل أو يسمح سجل تدقيق موجود بالفعل — تطبيق حرفي للبند 5.3 في CLAUDE.md ("يجب ألا يكون قابلاً للتعديل أو الحذف من قبل المستخدم أبداً").

bed-history.service.js بُني بعد كده كغلاف رفيع فقط حول
audit.service.writeAuditLog() — مفيهوش أي آلية تسجيل منفصلة،
بالظبط زي ما طلب قسم "Added After Phase 2 Review" في مستند المرحلة.

الخطوة 5: نقاط الوصول لكل من صاحب المبنى (Owner-Facing) (CRUD)

كل endpoint خاص بمبنى/شقة/سرير موجود، بيتحقق أولاً من وجود المورد (404 لو مش موجود)، وبعدين بيستخدم دالة ownershipScoping() الموجودة بالفعل في auth.middleware.js (من المرحلة الأولى) للتأكد إن req.user.ownerId يطابق owner_id بتاع المورد — لو مفيش تطابق، بيرجع 403 صراحةً. الفحص ده موجود على كل فعل يلمس مورد موجود بالفعل (قراءة/تعديل/حذف)، مش بس على الإنشاء.

الخطوة 6: نقطة القراءة المتداخلة (building → apartments) (beds)

GET /api/buildings/:buildingId بيرجّع المبنى مع كل شقيقه وأسرّة كل شقة في استجابة واحدة. لتفادي مشكلة N+1 (البند 4.4 في CLAUDE.md)، التنفيذ بيعمل استعلامين فقط بغض النظر عن عدد الشقق أو الأسرّة: استعلام واحد لكل شقق المبنى، واستعلام واحد تاني لكل أسرّة الشقق دي مجتمعة (عبر \$in على مصفوفة معرّفات الشقق) — بدل استعلام منفصل لكل شقة على حدة.

الخطوة 7: حساب الإشغال (Occupancy Calculation)

منطق الحساب اتخط في bed.service.js بالظبط زي ما طلب نص المرحلة ("Occupancy calculation logic lives here")، عن طريق دالة computeOccupancy() بتستخدم MongoDB Aggregation (group \$ على حقل status) — استعلام واحد بيرجّع أعداد الأسرّة لكل حالة، سواء على مستوى المبنى بالكامل أو شقة واحدة بس. building.service.js بينادي الدالة دي عن طريق bed.service، مش بيكرر منطق الحساب.

الخطوة 8: اختبار سلامة الهرم (Hierarchy Integrity)

تم تطبيق قاعدة تقييد الحذف (restrict) بدل الحذف المتسلسل (cascade) في الثلاث مستويات؛ لا يمكن حذف مبنى لسه فيه شقق، ولا حذف شقة لسه فيها أسرة، ولا حذف سرير حالته مش available. القرار وسببه موضحين في قسم "القرارات التقنية" و"الانحراف عن الخطة".

الخطوة 9: ربط endpoint تغيير حالة الـ KYC بموديول التدقيق (Retrofit)

`kyc.service.updateVerificationStatus()` بقت تنادي `auditService.writeAuditLog()` بعد كل تحديث لحالة التحقق (`/verified` `rejected`)، ويحتفظ بالـ `after_state/before_state` (حالة التحقق قبل وبعد). الحقول القديمة `reviewed_at/reviewed_by` على سجل الـ KYC نفسه **اتسابت زي ما هي** للاستعلام السريع، لكن سجل التدقيق دلوقتي هو المصدر الرسمي — بالظبط زي ما طلب قسم "Added After Phase 2 Review".

أي قرارات تقنية اتأخذت أثناء التنفيذ

1. نسخ `owner_id` و `building` على مستوى الشقة **والسرير** بدل الاعتماد على `populate()` متسلسل في كل عملية تحقق من الملكية أو حساب إشغال. القيمة دي بتتحدد مرة واحدة وقت الإنشاء من المورد الأب المُتحقّق من ملكيته بالفعل، وميتغيرش بعد كده مباشرة من أي طلب خارجي — فمفيش خطر إنها "تنحرف" عن المالك الحقيقي. القرار ده تطبيق مباشر للبند 4.4 في CLAUDE.md (تجنّب أنماط N+1)، وهو نفس المنطق اللي استُخدم في نقطة القراءة المتداخلة (الخطوة 6 أعلاه).
2. إضافة `pagination.util.js` **مشترك** — مش من الملفات المذكورة صراحةً في مستند المرحلة، لكنه نتيجة مباشرة لتطبيق قاعدة المستند نفسها (كل endpoint بيرجع قائمة لازم يكون فيه Pagination من أول يوم، البند 4.2 في CLAUDE.md). تكرار نفس منطق الـ parsing في أربع موديولات (`buildings/apartments/beds/audit`) كان هيبقى مصدر أخطاء متكرر، فاتعمل ملف مشترك واحد بدل أربع نسخ.

3. قيود الحذف (Restrict) بدل الحذف المتسلسل (Cascade) في
الثلاث مستويات — قرار أمان افتراضي (البند 3.11: أقل صلاحية ممكنة
كافتراضي). حذف مبنى يسمح شققه وأسرتَه تلقائيًا كان ممكن يبقى مريح
لصاحب المبنى، لكنه خطر لو حصل بالغلط، خصوصًا مع دخول بيانات حجوزات
حقيقية في المرحلة الرابعة. القرار: أي حذف لازم يبدأ من أسفل الهرم لفوق
(سرير فاضي الأول، بعدين شقة فاضية، بعدين مبنى فاضي).

4. استخدام `BED_STATUS` الموجود بالفعل في
`constants.config.js` بدل القيم المذكورة في تعليق هيكل
المجلدات بمستند المرحلة — تفاصيل السبب في قسم الانحراف التالي.

5. استخدام حالة السرير نفسها (`status`) كمؤشر لتقييد الحذف، بدل
الاعتماد على "إيجارات نشطة" كما هو مذكور حرفيًا في نص
المرحلة — لأن مودول الإيجارات (Rentals) لسه مش موجود حتى المرحلة
الرابعة. تفاصيل كاملة في قسم الانحراف التالي.

الاختبارات اللي اتعملت والنتائج

تم إنشاء `tests/integration/buildings-apartments-beds.test.js` ويغطي:

- **CRUD كامل** للمباني/الشقق/الأسرة: إنشاء، قراءة (بما فيها القراءة المتداخلة
`(building→apartments→beds)`، تعديل، وحذف — مع التحقق من صحة
الحقول المطلوبة (422 عند نقصها).
- **اختبارات عزل الملكية الصريحة (Negative Tests)** — تطبيقًا للبند 6.3
في `CLAUDE.md`: صاحب مبنى B مرفوض صراحةً (403) عند محاولة قراءة/
تعديل/حذف مبنى صاحب A، وعند إنشاء شقة تحت مبنى صاحب A، وعند قراءة/
تعديل/حذف شقة أو سرير صاحب A مباشرة بالمعرف — بالإضافة لاختبار تحكم
إيجابي (positive control) يتأكد إن صاحب A نفسه يقدر يوصل لبياناته بشكل
طبيعي.
- **اختبار Pagination**: التأكد إن قائمة مباني صاحب معيّن بترجع بياناته هو بس
(مش بيانات صاحب ثاني)، وإن قوائم الشقق/الأسرة بتحترم `limit / page`
وبترجع `meta.pages / meta.total` صح.

- **اختبارات سلامة الهرم:** رفض حذف مبنى لسه فيه شقق (409)، رفض حذف شقة لسه فيها أسرة (409)، رفض حذف سرير حالته مش `available` (409)، ونجاح الحذف المتسلسل من الأسفل للأعلى بعد تفريغ كل مستوى.
- **اختبار حساب الإشغال:** التأكد إن الأعداد المرجوعة (`occupied`, `total`, `available`, `maintenance`) صحيحة مع خليط حالات مختلفة.
- **اختبار كتابة سجل التدقيق عند تغيير حالة السرير:** التأكد إن تغيير `status` يكتب سجل واحد في `audit_logs` بالفعل الصحيح (`bed_status_change`) وبحالة قبل/بعد صحيحة، وإن تعديل حقل ثاني (زي `room_label`) **ميكاتبش** أي سجل تدقيق (عشان نتأكد إن الكتابة بتحصل بس لما الحالة فعلاً بتتغير).
- **اختبار ربط الـ KYC بموديول التدقيق:** التأكد إن موافقة/رفض `super-admin` لـ `admin` يسجل `KYC` يكتب سجل `kyc_status_change` صحيح في `audit_logs`، وإنه قابل للاسترجاع عن طريق `endpoint` استعلام التدقيق (`GET /api/audit?entity_type=Kyc&entity_id=...`).
- **اختبارات حدود الأدوار (Role Guards):** رفض توكن طالب على إنشاء مبنى (403)، رفض الوصول من غير توكن على كل الـ `endpoints` الجديدة (401)، رفض توكن `owner` و `student` على `endpoint` استعلام التدقيق (403) — لـ `super-admin` (بس)، ونجاح `super-admin` في الوصول له.

نتيجة التشغيل الفعلي: تم التحقق من كل الاختبارات أعلاه فعليًا عن طريق GitHub Actions (سيرفر حقيقي مع MongoDB كـ `service container`، بالظبط زي المرحلتين الأولى والثانية) — **النتيجة: نجاح كامل (Status: Success)**.
 تشغيل رقم 5 على `main branch`, `a17ce51 commit`, ("phase3"). رابط التشغيل: <https://github.com/vdivcompany-cmd/Sakanify-backend/actions/runs/30689315259> — مدة التشغيل: دقيقة و43 ثانية.

قبل الدفع، تم التحقق من خلو كل الملفات الجديدة والمعدلة من أخطاء `syntax` (`node -c` على كل ملف)، والتأكد من تحميل شجرة الموديولات بالكامل بنجاح فعليًا (`require('./src/app.entry.js')` من غير أي خطأ) — للتأكد من عدم وجود مشاكل تبعية دائرية (`circular dependency`) بين الموديولات الجديدة قبل الاعتماد على GitHub Actions للتحقق النهائي. التشغيل الفعلي للاختبارات نفسها لم يتم محليًا في بيئة الـ `sandbox` بسبب نفس القيد الموثق سابقًا في ذاكرة المشروع (تعذر تحميل ثنائي MongoDB اللازم لـ `mongodb-memory-server`).

بسبب قيود شبكة الـ GitHub Actions — (sandbox ظل، كالعادة، الطريقة الوحيدة الموثوقة للتحقق الفعلي).

أي انحراف عن الخطة الأصلية

الانحراف الأول: قيم `status` الخاصة بالسريـر

نص المرحلة (في تعليق هيكل المجلدات داخل `phase-3-buildings` `apartments-beds.md`) يذكّر قيم `available/requested/confirmed/` `vacating` لحقل حالة السريـر. لكن `src/config/constants.config.js` — اللي كان موجود بالفعل في المستودع من مرحلة سابقة — يعرّف `enum` اسمه `BED_STATUS` بالقيم `available/pending/occupied/maintenance`، مع تعليق صريح بيوضح إنه اتصمم خصيصًا لدعم آلية القفل الذري (atomic locking) في المرحلة الرابعة.

القرار: تم استخدام `BED_STATUS` الموجود بالفعل، مش القيم المذكورة في نص المرحلة. **السبب:** لو اتبنى حقل `status` بقيم مختلفة عن الـ `enum` الجاهز، كان هيبقى فيه قاموسين متعارضين لنفس المفهوم في نفس المشروع — واحد في `bed.model.js` وواحد في `constants.config.js` — وده هيسبب تعارض حقيقي للمرحلة الرابعة اللي المفروض تعتمد على `BED_STATUS` بالتحديد لبناء آلية القفل الذري المختبرة تحت التزامن (Concurrency)، حسب البند 4.5 من `CLAUDE.md`. الالتزام بالـ `enum` الجاهز والمُعلّق عليه صراحةً كان القرار الأكثر أمانًا واتساقًا مع باقي المشروع.

الانحراف الثاني: معيار تقييد حذف السريـر ("إيجارات نشطة" غير موجودة بعد)

نص المرحلة بيطلب: *"block deletion of a building with active rentals"*. لكن مودول الإيجارات (Rentals) لسه `placeholder` فاضي تمامًا حتى المرحلة الرابعة — مفيش أي علاقة "إيجار نشط" ممكن يتحقق منها فعليًا دلوقتي.

القرار: بدل الانتظار للمرحلة الرابعة أو اختراع آلية مؤقتة، تم استخدام **حقل `status` بتاع السريـر نفسه** (المبني فعليًا في هذه المرحلة) كمؤشر بديل: سريـر حالته غير `available` (يعني `maintenance / occupied / pending`) يمثل نوع

من العلاقة النشطة، حتى من غير وجود سجل إيجار رسمي بعد، فمينفعش يتحذف. وبالمثل، شقة لسه فيها أسرة أو مبنى لسه فيه شقق ميتحذفوش. **السبب:** ده تطبيق لنفس منطق الانحراف الموثق في تقرير المرحلة الثانية (تأجيل endpoint اطلاع صاحب المبنى للمرحلة الرابعة لعدم وجود علاقة Rentals حقيقية بعد) — نفس المبدأ: منقدرش نبني تحقق حقيقي على علاقة مش موجودة، فبنستخدم أقرب إشارة موجودة فعليًا وموثقة بدل التخمين، تطبيقًا للبند 7.5 من CLAUDE.md.

الأثر على المرحلة الرابعة: لما موديول Rentals يتبني، التوصية إن قاعدة تقييد الحذف تتراجع وتتربط بوجود إيجار نشط فعلي بدل حالة السرير بس، مع الاحتفاظ بفحص حالة السرير كطبقة حماية إضافية.

لا يوجد انحراف في نطاق موديول Audit نفسه

موديول التدقيق اتبنى بالكامل زي ما طلب قسم "Added After Phase 2 Review" حرفيًا — بما في ذلك ربط `bed-history.service` كغلاف رفيع، وربط endpoint الـ KYC بأثر رجعي، والاحتفاظ بحقلي `reviewed_at / reviewed_by` على سجل الـ KYC نفسه.

الحالة النهائية للمرحلة

الحالة: مكتملة بالكامل ومُتحقق منها فعليًا (Fully Complete & Verified).

الكود جاهز بالكامل: موديولات `Buildings/Apartments/Beds/Audit`، إعادة ربط KYC، ملف الـ `pagination` المشترك، ملف الاختبارات، وربط كل الـ `routes` الجديدة في `app.entry.js`. تم الدفع (push) بنجاح إلى `origin/main` — `commit` `a17ce51` بعنوان "phase3" — وتشغيل `GitHub Actions` تلقائيًا عليه (workflow `Sakanify Backend — Tests`، تشغيل رقم `#5`، **النتيجة: نجاح كامل (Success)**، بيئة `MongoDB` حقيقية كخدمة (service container)، وليست محاكاة.

رابط التشغيل الفعلي على GitHub Actions: [/https://github.com/vdivcompany-cmd/Sakanify-backend/actions/runs/30689315259](https://github.com/vdivcompany-cmd/Sakanify-backend/actions/runs/30689315259)

بهذا تُغلق المرحلة الثالثة رسميًا. الهرم الكامل (Building → Apartment → Bed)
جاهز ومُختَبَر، آلية التدقيق الحقيقية شغالة ومربوطة بكل من الأسرّة وال KYC،
وحساب الإشغال جاهز للاستخدام من موديولات الاشتراكات (Phase 6) والإدارة
(Phase 7) لاحقًا — تمامًا كما هو مطلوب في تسليم المرحلة. المرحلة الرابعة
(محرك الحجز والقفل الذري) جاهزة للبدء عليها الآن، مع الأخذ في الاعتبار توصية
مراجعة قاعدة تقييد الحذف بمجرد بناء موديول Rentals، الموضّحة في قسم الانحراف
أعلاه.